Learning Objectives

At the end of the class you should be able to:

- explain the model of deterministic planning
- represent a problem using the STRIPs representation of actions.

State-space Search

- deterministic or stochastic dynamics
- fully observable or partially observable
- explicit states or features or individuals and relations
- static or finite stage or indefinite stage or infinite stage
- goals or complex preferences
- perfect rationality or bounded rationality
- flat or modular or hierarchical
- single agent or multiple agents
- knowledge is given or knowledge is learned
- reason offline or reason while interacting with environment

Classical Planning

- deterministic or stochastic dynamics
- fully observable or partially observable
- explicit states or features or individuals and relations
- static or finite stage or indefinite stage or infinite stage
- goals or complex preferences
- perfect rationality or bounded rationality
- flat or modular or hierarchical
- single agent or multiple agents
- knowledge is given or knowledge is learned
- reason offline or reason while interacting with environment

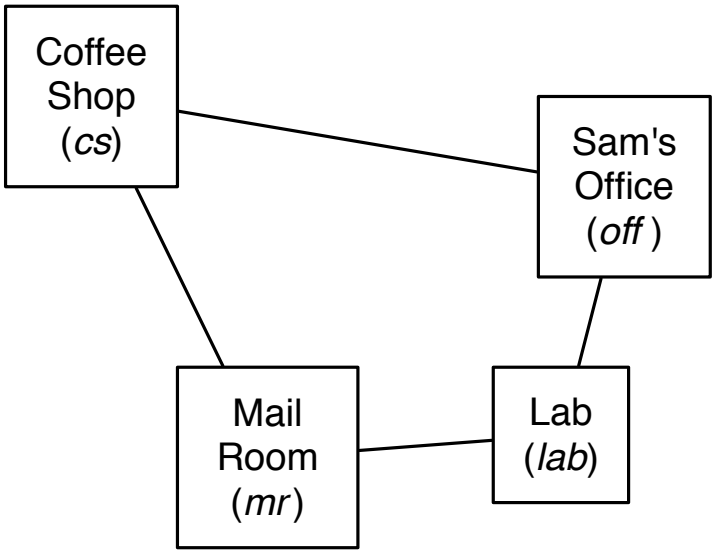
Planning

- Planning is deciding what to do based on an agent's ability, its goals, and the state of the world.
- Initial assumptions:
  - The world is deterministic. There are no exogenous events outside of the control of the robot that change the state of the world.
  - The agent knows what state it is in. (Fully observable)
  - Time progresses discretely from one state to the next.
  - Goals are predicates of states that need to be achieved.
- Planning is finding a sequence of actions to solve a goal.

Actions

- A deterministic action is a partial function from states to states.
- The preconditions of an action specify when the action can be carried out.
- The effect of an action specifies the resulting state.

Delivery Robot Example



Features:

RLoc – Rob's location
RHC – Rob has coffee
SWC – Sam wants coffee
MW – Mail is waiting
RHM – Rob has mail

Actions:

mc – move clockwise
mcc – move counterclockwise
puc – pickup coffee
dc – deliver coffee
pum – pickup mail
dm – deliver mail

Explicit State-space Representation

State	Action	Resulting State
$\langle lab, \neg rhc, swc, \neg mw, rhm \rangle$	mc	$\langle mr, \neg rhc, swc, \neg mw, rhm \rangle$
$\langle lab, \neg rhc, swc, \neg mw, rhm \rangle$	mcc	$\langle off, \neg rhc, swc, \neg mw, rhm \rangle$
$\langle off, \neg rhc, swc, \neg mw, rhm \rangle$	dm	$\langle off, \neg rhc, swc, \neg mw, \neg rhm \rangle$
$\langle off, \neg rhc, swc, \neg mw, rhm \rangle$	mcc	$\langle cs, \neg rhc, swc, \neg mw, rhm \rangle$
$\langle off, \neg rhc, swc, \neg mw, rhm \rangle$	mc	$\langle lab, \neg rhc, swc, \neg mw, rhm \rangle$
...	...	...

What happens when we also want to model its battery charge?
Want "elaboration tolerance".

STRIPS Representation

The state is a function from features into values. So it can be represented as a set of feature-value pairs.
For each action:

- precondition a set of feature-value pairs that must be true for the action to be carried out.
- effect a set of a feature-value pairs that are made true by this action.

STRIPS assumption: every feature not mentioned in the effect is unaffected by the action.

Example STRIPS representation

Pick-up coffee (puc):

- precondition: $\{RLoc = cs, RHC = False\}$. "The robot needs to be at the coffee shop and not holding coffee in order to pick up coffee."
- effect: $\{RHC = True\}$. "After the robot picks up coffee, it is holding coffee. Nothing else has changed."

Deliver coffee (dc):

- precondition: $\{Rloc = off, RHC = True\}$ "The robot needs to be at the office and holding coffee in order to deliver coffee."
- effect: $\{RHC = False, SWC = False\}$ "After the robot delivers coffee, it is not holding coffee, and Sam no longer wants coffee. Nothing else has changed."

Deterministic Planning

Given:

- A description of the effects and preconditions of the actions
- A description of the initial state
- A goal to achieve

find a sequence of actions that is possible and will result in a state satisfying the goal.

Deterministic Planning

Given:

- A description of the effects and preconditions of the actions
- A description of the initial state
- A goal to achieve

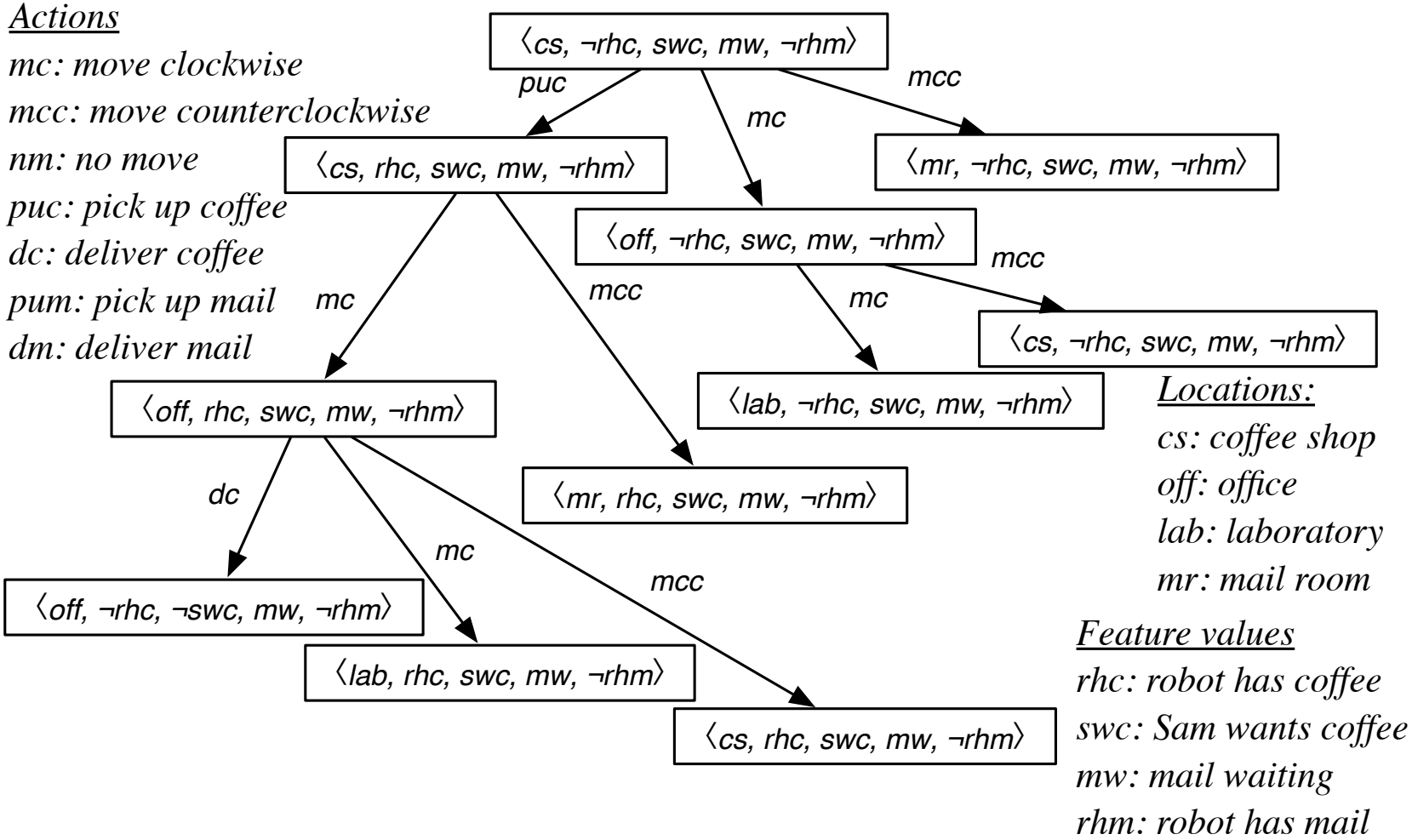
find a sequence of actions that is possible and will result in a state satisfying the goal.

Forward Planning

Idea: search in the state-space graph.

- The nodes represent the states
- There is an arc $\langle s, s' \rangle$ labeled with action A if
  - A is an action that can be carried out in state s and
  - s' is the state resulting from doing A in state s
- A plan is a path from the state representing the initial state to a state that satisfies the goal.

Example state-space graph



Actions

mc: move clockwise
mcc: move counterclockwise
nm: no move
puc: pick up coffee
dc: deliver coffee
pum: pick up mail
dm: deliver mail

Locations:

cs: coffee shop
off: office
lab: laboratory
mr: mail room

Feature values

rhc: robot has coffee
swc: Sam wants coffee
mw: mail waiting
rhm: robot has mail

Forward planning representation

- The search graph can be constructed on demand: you only construct reachable states.
- If you want a cycle check or multiple path-pruning, you need to be able to find repeated states.
- There are a number of ways to represent states:
  - As a map from features into their values
  - As a path from the start state

Improving Search Efficiency

Forward search can use domain-specific knowledge specified as:

- a heuristic function that estimates the cost from a complete state description to a goal.
- domain-specific pruning of neighbors:
  - don't pick-up coffee unless Sam wants coffee.
  - unless the goal involves time constraints, don't do a "no move" action.
  - don't go to the coffee shop unless "Sam wants coffee" is part of the goal and Rob doesn't have coffee (maybe not)

Regression Planning

Idea: search backwards from the goal description: nodes correspond to subgoals, and arcs to actions.
A subgoal is an assignment of values to some features.

Search problem:

- Nodes are subgoals
- There is an arc $\langle g, g' \rangle$ labeled with action A if
  - A achieves one of the assignments in g
  - g' is a proposition that must be true immediately before action A so that g is true immediately after.
- The start node is the goal to be achieved.
- $goal(g)$ is true if g is a proposition that is true of the initial state.

Defining nodes and arcs

- A node g can be represented as a set of assignments of values to variables:

  $$[X_1 = v_1, \ldots, X_n = v_n]$$

  This is a set of assignments you want to hold.
- The last action achieves one of the $X_i = v_i$, and does not achieve $X_j = v'_j$ where v'_j is different to v_j.
- The neighbor of g along arc A must contain:
  - The prerequisites of action A
  - All of the elements of g that were not achieved by A

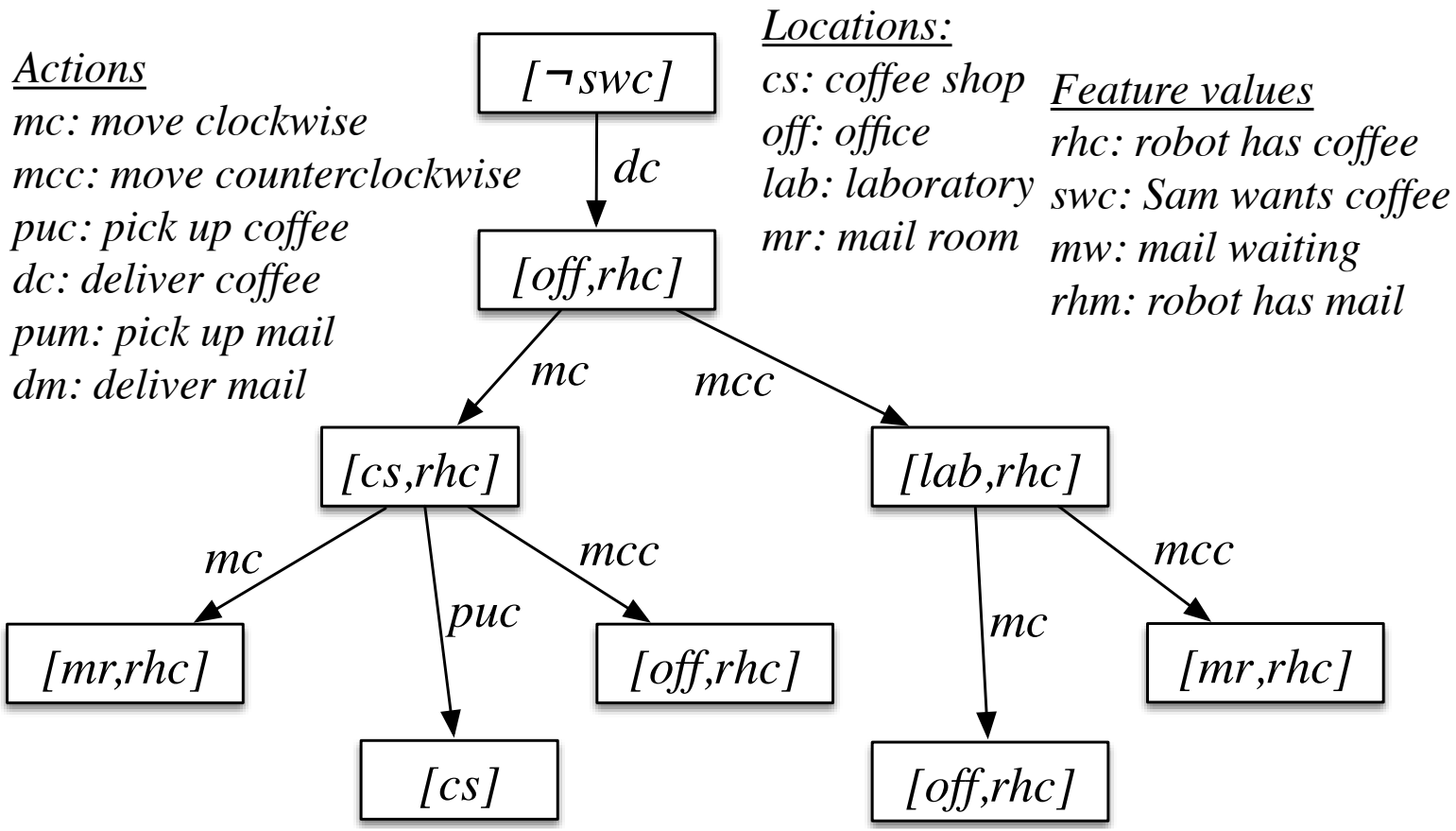
  it must be consistent = have at most one value for each feature.

Formalizing arcs using STRIPS notation

$\langle g, g' \rangle$ is an arc labeled with action A where g is $[X_1 = v_1, \ldots, X_n = v_n]$ and A is an action, if

- $\exists i\ X_i = v_i$ is on the effects list of action A
- $\forall j\ X_j = v'_j$ is not on the effects list for A, where $v'_j \neq v_j$
- $g' = \mathit{preconditions}(A) \cup \{X_k = v_k \in g : X_k = v_k \notin \mathit{effects}(A)\}$ if it is consistent

Regression example



Loop detection and multiple-path pruning

- Goal G_1 is simpler than goal G_2 if G_1 is a subset of G_2.
  - It is easier to solve $[cs]$ than $[cs, rhc]$.
- If you have a path to node N have already found a path to a *simpler* goal, you can prune the path N.

Improving Efficiency

- You can define a heuristic function that estimates how difficult it is to solve a goal from a state.
  A heuristic function defined the cost of getting from a state to a (sub)goal. This is the same as a heuristic for the forward planner.
- You can use domain-specific knowledge to remove impossible goals, e.g.
  - It is often not obvious from an action description to conclude whether an agent can hold multiple items at any time.

Comparing forward and regression planners

- Which is more efficient depends on:
  - The branching factor
  - How good the heuristics are
- Forward planning is unconstrained by the goal (except as a source of heuristics).
- Regression planning is unconstrained by the initial state (except as a source of heuristics)

Planning as a CSP

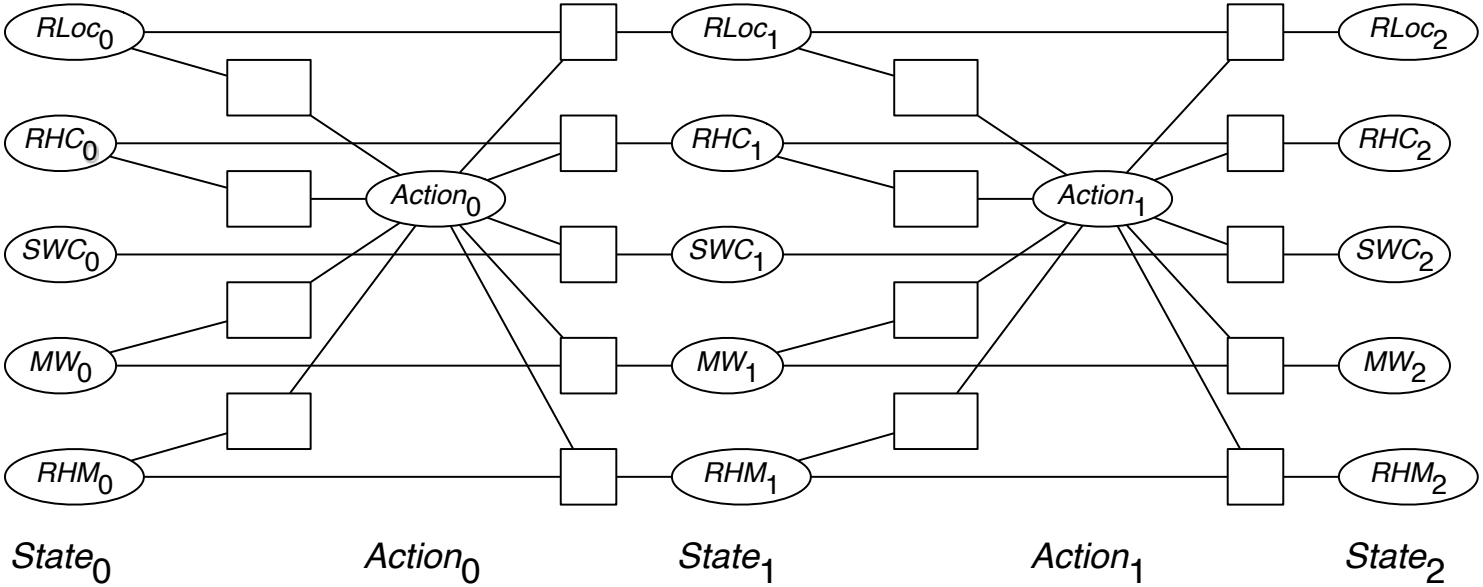
- In forward planning, the nodes considered are constrained to be reachable, even if they don't lead to goal.
- In regression planning, the nodes considered are constrained to be ones from which we can achieve the goal, even if they are not reachable.
- When representing planning as a CSP, we can constrain the states by the starting state and the goal.
  . . . but we can only do this if we know the number of steps.
- Search over planning horizons (number of time steps).
- For each planning horizon, create a CSP constraining possible actions and features

CSP Variables

Choose a planning horizon k.

- Create a variable for each state feature and each time from 0 to k.
- Create a variable for the action for each time in the range 0 to $k - 1$.

CSP for Delivery Robot for a planning horizon of 2



$RLoc_i$ — Rob's location
RHC_i — Rob has coffee
SWC_i — Sam wants coffee
MW_i — Mail is waiting
RHM_i — Rob has mail

$Move_i$ — Rob's move action
PUC_i — Rob picks up coffee
$DelC$ — Rob delivers coffee
PUM_i — Rob picks up mail
$DelM_i$ — Rob delivers mail

Constraints

- precondition constraints between state variables at time t and action variable at time t, specify constraints on what actions are available from a state.
- effect constraints between state variables at time t, action variable at time t and state variables at time $t+1$ constrain the resulting state to be one that satisfies the effects.
- frame constraints among state variables at time t, action variables at time t, and state variables at time $t+1$ for values of variables that do not change.
- initial state constraints that are usually domain constraints on the initial state (at time 0).
- goal constraints that constrains the final state to be a state that satisfies the goals that are to be achieved.

Example Constraints

precondition constraint $RLoc_i = cs \leftarrow Action_i = puc$
is violated when $RLoc_i \neq cs \wedge Action_i = puc$

effect constraint $rhc_{i+1} \leftarrow Act_i = puc$
is violated when $RHC_{i+1} = false \wedge Act_i = puc$.

frame constraint Rob has mail at any time if it had mail before and the action wasn't to pickup mail or deliver mail:

$$RHM_{i+1} = RHM_i \leftarrow Act_i \notin \{pum, dm\}$$

violated when
$RHM_{i+1} \neq RHM_i \wedge Act_i \neq pum \wedge Act_i \neq dm$